DATA QUALITY ENGINEERING

Data Quality
Rules Catalog

Document 4 of 5 Enterprise Data Governance & Quality Programme
Rules in Catalog DQR-001 through DQR-020 (20 active rules)
Platform Scope Snowflake + Databricks / Delta Live Tables
Aligned to DB Coding & Naming Standards v2.0 | Case Study | DQ Test Plan v1
Catalog Owner Data Quality Lead · Review: Data Stewards · Approval: Data Owners

1. Purpose & Scope of the Rules Catalog

The Data Quality Rules Catalog is the single authoritative registry for every data quality rule operating on the enterprise data platform. It translates the DQ framework principles from the Database Coding & Naming Standards (v2.0) and the validated test cases from the DQ Test Plan into a living operational record: who owns each rule, what it checks, what it expects to find, what it does when it finds a problem, and when it was last reviewed.

This is not a reference document. It is a governance artefact. Every rule in this catalog is active in production, mapped to an enforcement tool, assigned to a named Data Steward, and subject to a defined review cycle. Rules without these properties are non-compliant with the framework and must either be remediated or formally deprecated.

What This Catalog Governs
‣ 20 active rules — DQR-001 through DQR-020 — spanning 5 DQ dimensions across 5 datasets ‣ Rule lifecycle: Draft → Steward Review → Active → Deprecated (with full audit trail)
‣ Enforcement tool mapping: every rule maps to dbt, GE, Snowflake Task, DLT, or CI/CD
‣ Exception registry: rules with approved exceptions, compensating controls, and expiry dates
‣ Intake process: formal template and approval workflow for new rule proposals
‣ Cross-reference: each rule links to its test case (TC-XXX) in the DQ Test Plan ‣ Review cadence: Quarterly (Critical/High), Biannual (Medium), Annual (Low)

2. Catalog Overview — All 20 Rules at a Glance

The table below provides the complete registry view. Each rule is linked to its full card in Section 4. The catalog is sorted by severity, then by rule ID. This view is the primary reference for the nightly DQS dashboard and the quarterly governance review.

Rule ID	Rule Name	Table	Dimension	Sev.	Status	Tool	Test	Review
DQR-001	PK Not Null	fact_order	Completeness	**Critical**	**Active**	dbt + GE	TC-001	Q
DQR-002	PK Uniqueness	fact_order	Uniqueness	**Critical**	**Active**	dbt + GE	TC-002	Q
DQR-003	FK: order→customer	fact_order	Ref. Integrity	**Critical**	**Active**	dbt + GE	TC-003	Q
DQR-009	Freshness SLA 1hr	fact_order	Timeliness	**Critical**	**Active**	SF Alert	TC-009	Q

DQR-013	Audit Column Completeness	All managed tables	Completeness	**Critical**	**Active**	dbt + CI	TC-013	Q
DQR-020	Schema Drift Detection	All Tier 1+2 tables	Schema	**Critical**	**Active**	dbt + SF	TC-020	Q
DQR-004	Allocation Pct Sum ≤ 100%	fact_resource_alloc	Validity	**High**	**Active**	dbt	TC-004	Q
DQR-005	Customer Name Completeness	dim_customer	Completeness	**High**	**Active**	dbt + GE	TC-005	Q
DQR-006	Order Amount Non-Negative	fact_order	Validity	**High**	**Active**	dbt + GE	TC-006	Q
DQR-012	Order Status Domain	fact_order	Validity	**High**	**Active**	dbt + GE	TC-012	Q
DQR-014	Record Hash Uniqueness	fact_order, dim_cust	Uniqueness	**High**	**Active**	dbt + SF Task	TC-014	Q
DQR-015	Volume Anomaly Detection	fact_order	Timeliness	**High**	**Active**	SF Task + GE	TC-015	Q
DQR-016	Surrogate Key Integrity	dim_customer	Uniqueness	**High**	**Active**	dbt	TC-016	Q
DQR-018	SCD Currency Flag	dim_customer	Validity	**High**	**Active**	dbt + SF Task	TC-018	Q
DQR-019	Staging→Curated Reconciliation	stg_order→fact_order	Consistency	**High**	**Active**	dbt + SF Task	TC-019	Q
DQR-007	Order Date Not Future	fact_order	Validity	**Medium**	**Active**	dbt + GE	TC-007	B
DQR-008	Boolean Not Null	dim_customer	Completeness	**Medium**	**Active**	dbt + GE	TC-008	B
DQR-010	Country Code in Ref List	dim_customer	Validity	**Medium**	**Active**	dbt	TC-010	B
DQR-011	Zero-Amount Confirmed Order	fact_order	Validity	**Medium**	**Active**	dbt	TC-011	B
DQR-017	Empty String Detection	dim_customer + orders	Completeness	**Medium**	**Active**	dbt	TC-017	B

Review cadence key: Q = Quarterly | B = Biannual | A = Annual · Severity thresholds per framework Section 9.1

3. Rule Lifecycle, Governance & Operations

3.1 Rule Lifecycle States

Every rule in the catalog exists in one of four states. State transitions require approval at the level defined below. No rule moves to Active without a passing test case in the DQ Test Plan.

State	Definition	Entry Requirement	Exit Requirement	Approval
Draft	Rule authored but not yet reviewed	Engineer proposes rule with SQL and severity	Steward review scheduled	Rule Author
In Review	Steward is validating logic and severity	Draft complete; SQL tested in DEV environment	Steward sign-off; test case added to Test Plan	Data Steward
Active	Rule is live in production enforcement	Test case passing in QA; Steward approved; tool configured	Deprecation request raised	Data Steward (Medium/Low) | Data Owner (High/Critical)
Deprecated	Rule is retired and no longer enforced	Business rule obsoleted; superseded by new rule; dataset retired	Logged in audit; consumers notified	Data Owner + Data Steward

3.2 Rule Activation Process

New rules follow a four-step activation process. No rule bypasses this process — including rules raised as urgent incident responses. An urgent rule may be fast-tracked (24-hour Steward review instead of 5 business days) but must still complete all steps.

- Step 1 — Proposal. Engineer or Steward completes the New Rule Intake Template (Section 5). Template requires: rule name, target table, validation SQL, severity classification, proposed tool, and business justification.
- Step 2 — DEV Validation. Engineer runs the SQL against DEV synthetic data. Both positive (clean data passes) and negative (injected violations detected) test scenarios must execute correctly. Results documented in the Test Plan.
- Step 3 — Steward Review. Data Steward validates: severity is correctly classified, SQL logic is correct, enforcement tool is appropriate, and the rule does not duplicate an existing rule. Review SLA: 5 business days (standard) or 24 hours (urgent).
- Step 4 — Owner Approval & Activation. For Critical and High rules: Data Owner approval required. For Medium and Low: Steward approval is sufficient. On approval, rule is activated in the enforcement tool and status set to Active in this catalog.

3.3 Deprecation Process

Rules are deprecated — not deleted. Deprecated rules remain in the catalog with their history intact. The audit trail for why a rule was active, for how long, and why it was retired is a governance requirement.

- Deprecation triggers: dataset retired, rule superseded by a broader rule, business process eliminated, or rule logic proven incorrect.
- Deprecation requires: Data Steward recommendation + Data Owner approval + 14-day consumer notice for Critical/High rules + audit_log entry.
- Deprecated rules are moved to Section 6 of this catalog. Their SQL is preserved. Their test cases are marked Deprecated in the Test Plan.

3.4 Exception Process

A rule exception is a time-limited approval to allow known violations without triggering the standard fail action. Exceptions are for business anomalies with a defined remediation timeline — not for permanently ignoring a rule.

Exception Field	Requirement
Rule being excepted	DQR-XXX — full rule ID
Business justification	Specific reason the rule cannot currently be met
Affected records	Scope of the exception — how many records, which batch IDs
Compensating control	What alternative measure mitigates the risk during the exception period
Duration	Maximum 90 days per framework Section 3.4. No auto-renewal.
Approval	Data Steward (Medium/Low) | Data Owner + Steward (High/Critical)
Expiry action	Expired exceptions without renewal trigger an automatic remediation Jira ticket

3.5 Review Cadence & Ownership Matrix

Every active rule has a named Data Steward responsible for its ongoing accuracy and a defined review cycle. At each review, the Steward confirms: the SQL logic is still correct for the current schema, the severity is still appropriate, the threshold is still valid, and no new edge cases have emerged.

Severity	Review Frequency	Reviewer	Output	Escalation if Overdue
Critical	Quarterly	Data Steward + Data Owner	Updated rule card, version bump, Jira closed	Governance Lead notified; rule flagged non-compliant
High	Quarterly	Data Steward	Updated rule card if changed; no-change confirmation	Governance Lead notified

Medium	Biannual	Data Steward	Review note logged in catalog	Jira ticket auto-created
Low	Annual	Data Steward	Review note logged	No auto-escalation

3.6 DQS Dimension Contribution Map

Each rule contributes to one or more dimensions of the Data Quality Score composite. The map below shows which rules feed which DQS dimension — essential for understanding why the DQS moves when specific rules fail.

DQS Dimension	Weight	Contributing Rules	Dataset Coverage
Completeness	30%	DQR-001, 005, 008, 013, 017	fact_order, dim_customer, all managed tables
Uniqueness	20%	DQR-002, 014, 016	fact_order, dim_customer
Validity	20%	DQR-004, 006, 007, 010, 011, 012, 018	fact_order, dim_customer, fact_resource_allocation
Timeliness	20%	DQR-009, 015	fact_order
Referential Integrity	10%	DQR-003, 019	fact_order → dim_customer, stg_order → fact_order
Schema Compliance	N/A	DQR-013, 020 (pipeline gate — not DQS score)	All Tier 1 + 2 tables

Schema compliance rules (DQR-013, 020) are pipeline gates — they halt execution rather than contributing to a score. A halted pipeline produces no DQS output for that run.

4. Full Rule Cards — DQR-001 through DQR-020

Each rule card below contains the complete specification: metadata, business rule definition, validation SQL, pass condition, fail action, and any version notes. Rule cards are the authoritative reference for Data Custodians implementing enforcement and Data Stewards conducting quarterly reviews.

DQR-001 Primary Key Not Null

fact_order | marts / Gold | Tier 1

CRITICAL
Status: Active
v1.0

DQ Dimension	Completeness	**Test Case Ref**	TC-001
Primary Tool	dbt: not_null	**Secondary Tool**	GE: expect_column_values_to_not_be_null | Snowflake Task
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 7.3 — Data Health Rules | Section 5 — Data Type: _id NOT NULL	**Tags**	pk, completeness, fact_order, tier-1

Business Rule	Every row in fact_order must have a non-null order_id. A null primary key is unindexable, untraceable, and makes all downstream joins undefined. Zero null tolerance — not even one record is acceptable.
Pass Condition	**SELECT COUNT(*) FROM fact_order WHERE order_id IS NULL → returns 0**
Fail Action	**Halt pipeline immediately. Page on-call Data Steward via PagerDuty. Log to governance.audit_log. Open P1 Jira ticket. Block all downstream mart refreshes.**

```
-- DQR-001: Primary key null check
SELECT COUNT(*) AS null_pk_count
FROM   commerce_prod.marts.fact_order
WHERE  order_id IS NULL;

-- Expected: null_pk_count = 0
-- Threshold: any value > 0  =  CRITICAL FAILURE
```

DQR-002 Primary Key Uniqueness

fact_order | marts / Gold | Tier 1

CRITICAL
Status: Active
v1.0

DQ Dimension	Uniqueness	**Test Case Ref**	TC-002

Primary Tool	dbt: unique	**Secondary Tool**	GE: expect_column_values_to_be_unique | Snowflake Task
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 7.3 — 0% duplicate PK tolerance | Section 10.1 — dbt unique test mandatory	**Tags**	pk, uniqueness, fact_order, tier-1, deduplication

Business Rule	Each order_id must appear exactly once in the curated fact_order table. Duplicate primary keys corrupt every COUNT DISTINCT, SUM, and JOIN that downstream consumers execute. The deduplication ROW_NUMBER strategy in stg_order should prevent this from ever reaching the curated layer — this rule is the final guard.
Pass Condition	**GROUP BY order_id HAVING COUNT(*) > 1 → returns 0 rows**
Fail Action	**Halt pipeline. Open P1 ticket. Trigger deduplication re-run on stg_order. Notify Data Owner. Log root cause to audit_log.**
Notes	Complement with DQR-014 (record_hash uniqueness) for a second-layer uniqueness guarantee.

```
-- DQR-002: Duplicate primary key detection
SELECT   order_id,
         COUNT(*) AS dup_count
FROM     commerce_prod.marts.fact_order
GROUP BY order_id
HAVING   COUNT(*) > 1
ORDER BY dup_count DESC;

-- Expected: 0 rows returned
-- Impact metric (include in incident ticket):
SELECT COUNT(*) AS affected_order_ids,
       SUM(cnt - 1) AS excess_records
FROM ( SELECT order_id, COUNT(*) AS cnt
       FROM commerce_prod.marts.fact_order
       GROUP BY order_id HAVING COUNT(*) > 1 ) dup;
```

DQR-003 FK Integrity — fact_order → dim_customer

fact_order | marts / Gold | Tier 1

CRITICAL
Status: Active
v1.1

DQ Dimension	Referential Integrity	**Test Case Ref**	TC-003
Primary Tool	dbt: relationships	**Secondary Tool**	GE: custom SQL expectation | Snowflake Task
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q3 (Quarterly)

Framework Refs	Section 7.3 — FK: 100% referential integrity | Section 10.1 — dbt relationships test	**Tags**	fk, referential-integrity, fact_order, dim_customer, tier-1

Business Rule	Every customer_id in fact_order must resolve to a valid record in dim_customer. Orphaned order records produce invisible nulls in all customer-dimension joins, silently corrupting customer lifetime value, segmentation, and churn metrics.
Pass Condition	**LEFT JOIN dim_customer WHERE c.customer_id IS NULL → returns 0 rows**
Fail Action	**Alert Data Steward. Block downstream mart_sales_summary and mart_customer refresh. Quarantine orphaned records to a staging quarantine table. Investigate source of orphan: deleted dim record or late-arriving fact.**
Notes	v1.1: Added quantified impact query (orphan_pct) for DQS Referential Integrity dimension scoring.

```
-- DQR-003: Orphan FK detection
SELECT  o.order_id,
        o.customer_id,
        o.order_date,
        o.audit_batch_id
FROM    commerce_prod.marts.fact_order  AS o
LEFT JOIN commerce_prod.marts.dim_customer AS c
       ON o.customer_id = c.customer_id
WHERE   c.customer_id IS NULL;

-- Expected: 0 rows

-- Quantified impact for DQS scoring:
SELECT ROUND(COUNT(*) * 100.0 / NULLIF(
       (SELECT COUNT(*) FROM commerce_prod.marts.fact_order),0),4)
       AS orphan_pct
FROM   commerce_prod.marts.fact_order AS o
LEFT JOIN commerce_prod.marts.dim_customer AS c
       ON o.customer_id = c.customer_id
WHERE  c.customer_id IS NULL;
```

DQR-004 Allocation Percentage Sum ≤ 100%

fact_resource_allocation | marts / Gold | Tier 2

HIGH
Status: Active
v1.0

DQ Dimension	Validity	**Test Case Ref**	TC-004
Primary Tool	dbt: custom SQL test	**Secondary Tool**	GE: custom grouped expectation
Data Owner	Operations Domain Lead	**Data Steward**	Ops Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q4 (Quarterly)
Framework Refs	Section 5 — _pct: NUMBER(5,4), stored 0–1 decimal | Section 8 — Rules Catalog pattern	**Tags**	validity, business-rule, allocation, fact_resource_allocation, tier-2

Business Rule	The sum of allocation_pct values grouped by resource_id must not exceed 1.0000 (100%). Over-allocation is a business logic violation — a resource cannot be allocated to more than 100% of its capacity. The _pct suffix mandates NUMBER(5,4) storage as a decimal (0.15 = 15%), not an integer.
Pass Condition	**GROUP BY resource_id HAVING SUM(allocation_pct) > 1.0000 → returns 0 rows**
Fail Action	**Quarantine all rows for affected resource_ids. Alert Data Owner. Create High Jira ticket. Do not block upstream pipeline — but block downstream capacity dashboard refresh until resolved.**
Notes	SUM exactly 1.0000 is valid (fully allocated). Only > 1.0000 is a violation. Partial allocations (< 1.0) are permitted.

```
-- DQR-004: Over-allocation detection
SELECT  resource_id,
        ROUND(SUM(allocation_pct), 4)  AS total_allocation,
        COUNT(*)                        AS allocation_rows
FROM    commerce_prod.marts.fact_resource_allocation
GROUP BY resource_id
HAVING  SUM(allocation_pct) > 1.0000
ORDER BY total_allocation DESC;

-- Expected: 0 rows
-- Boundary: SUM = 1.0000 exactly is VALID — do not flag
```

DQR-005 Customer Name Completeness

dim_customer | marts / Gold | Tier 1

HIGH
Status: Active
v1.0

DQ Dimension	Completeness	**Test Case Ref**	TC-005
Primary Tool	dbt: not_null	**Secondary Tool**	GE: expect_column_values_to_not_be_null
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 7.1 — Completeness: % non-null mandatory fields | Section 4.4 — _name: VARCHAR non-null	**Tags**	completeness, dim_customer, customer_name, tier-1

Business Rule	customer_name must be populated for every active customer record. A null name breaks customer-facing reporting, personalisation logic, and regulatory disclosure requirements where customer identification is mandatory. Pair with DQR-017 (empty string detection) for full coverage.
Pass Condition	**COUNT(*) WHERE customer_name IS NULL → returns 0**
Fail Action	**Alert Data Steward. Log to DQ incident table. Do not halt pipeline — flag records with a dq_completeness_flag column. Trigger source-system investigation for null name origin.**

Notes	DQR-017 covers empty string and whitespace-only names — both rules are required together for full completeness coverage.

```
-- DQR-005: Customer name completeness with percentage
SELECT
    COUNT(*)                          AS total_customers,
    SUM(CASE WHEN customer_name IS NULL THEN 1 ELSE 0 END)
                                      AS null_name_count,
    ROUND(
        SUM(CASE WHEN customer_name IS NULL THEN 1 ELSE 0 END)
        * 100.0 / NULLIF(COUNT(*), 0), 4
    )                                 AS null_name_pct
FROM commerce_prod.marts.dim_customer;

-- Expected: null_name_count = 0
-- DQS impact: null_name_pct feeds the Completeness dimension weight
```

DQR-006 Order Amount Non-Negative

fact_order | marts / Gold | Tier 1

HIGH
Status: Active
v1.0

DQ Dimension	Validity	**Test Case Ref**	TC-006
Primary Tool	dbt: expression_is_true (order_amount > 0)	**Secondary Tool**	GE: expect_column_values_to_be_between(min=0.01)
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 5 — _amount: NUMBER(18,4), CHECK >= 0 | Section 8 — Rules Catalog	**Tags**	validity, fact_order, order_amount, financial, tier-1

Business Rule	order_amount must be strictly non-negative for all rows. A negative order amount is not a refund record — refunds are captured in a separate fact_refund table. A negative value here represents either a source system defect or a misrouted refund record, and will corrupt all revenue SUM aggregations.
Pass Condition	**COUNT(*) WHERE order_amount < 0 OR order_amount IS NULL → returns 0**
Fail Action	**Quarantine all rows with order_amount < 0 to staging.quarantine_negative_amounts. Alert Steward. Pipeline continues for clean records. Raise ticket to investigate source system.**
Notes	Zero-amount orders are governed by DQR-011 (separate rule). This rule targets strictly negative values only.

```
-- DQR-006: Negative and null order amount detection
SELECT  order_id,
        order_amount,
        order_status_code,
        source_system_id
```

```
FROM    commerce_prod.marts.fact_order
WHERE   order_amount < 0
   OR   order_amount IS NULL
ORDER BY order_amount ASC;

-- Expected: 0 rows
-- Note: order_amount = 0.00 is governed by DQR-011 separately
```

DQR-007 Order Date Not Future

fact_order | marts / Gold | Tier 1

MEDIUM
Status: Active
v1.0

DQ Dimension	Validity	**Test Case Ref**	TC-007
Primary Tool	dbt: expression_is_true (order_date <= current_date())	**Secondary Tool**	GE: expect_column_max_to_be_between(max_value='now')
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q4 (Biannual)
Framework Refs	Section 5 — _date: DATE, valid range via CHECK constraint | Section 8	**Tags**	validity, fact_order, order_date, temporal, tier-1

Business Rule	An order_date in the future is a data entry error or a timezone mishandling in the source system. Future dates cause orders to be invisible in 'current period' BI reports, inflating future-period forecasts with records that should appear today.
Pass Condition	**COUNT(*) WHERE order_date > CURRENT_DATE() → returns 0**
Fail Action	**Log warning to DQ incident table. Flag affected records with dq_validity_flag = 'FUTURE_DATE'. Do not quarantine — Steward review required before action. Create Medium Jira ticket.**
Notes	order_date = CURRENT_DATE() is valid (same-day orders). Only strictly future dates are violations.

```
-- DQR-007: Future order date detection
SELECT  order_id,
        order_date,
        DATEDIFF('day', CURRENT_DATE(), order_date) AS days_ahead
FROM    commerce_prod.marts.fact_order
WHERE   order_date > CURRENT_DATE()
ORDER BY order_date ASC;

-- Expected: 0 rows
-- Note: order_date = CURRENT_DATE() passes this rule
```

DQR-008 Boolean Not Null — is_active_flag

dim_customer | marts / Gold | Tier 1

MEDIUM
Status: Active
v1.0

DQ Dimension	Completeness	**Test Case Ref**	TC-008
Primary Tool	dbt: not_null + accepted_values([true, false])	**Secondary Tool**	GE: expect_column_values_to_not_be_null
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q4 (Biannual)
Framework Refs	Section 4.4 — _flag: BOOLEAN NOT NULL, DEFAULT required | Section 5 — Boolean constraint	**Tags**	completeness, dim_customer, boolean, flag, tier-1

Business Rule	is_active_flag must be non-null for all customer records. A null boolean is logically indeterminate — it is neither active nor inactive — making it impossible to correctly filter the active customer population. Framework Section 4.4 requires all _flag columns to use positive logic with a non-null default.
Pass Condition	**COUNT(*) WHERE is_active_flag IS NULL → returns 0**
Fail Action	**Apply DEFAULT TRUE to nulls as an immediate remediation. Log to DQ incident table. Create Medium ticket. Investigate upstream to prevent recurrence.**
Notes	Negative logic (is_not_active_flag) is a naming violation per Section 4.4 and is a separate CI/CD enforcement failure — not handled by this data rule.

```
-- DQR-008: Boolean completeness check
SELECT  COUNT(*) AS null_flag_count
FROM    commerce_prod.marts.dim_customer
WHERE   is_active_flag IS NULL;

-- Expected: null_flag_count = 0

-- Extended: ensure only TRUE/FALSE values present (no 0/1 integer bleed)
SELECT  DISTINCT is_active_flag
FROM    commerce_prod.marts.dim_customer;
-- Expected: only TRUE and FALSE appear
```

DQR-009 Freshness SLA — fact_order (Tier 1, 1-hour)

fact_order | marts / Gold | Tier 1

CRITICAL
Status: Active
v1.2

DQ Dimension	Timeliness	**Test Case Ref**	TC-009
Primary Tool	Snowflake Alert (10-min schedule)	**Secondary Tool**	dbt source freshness | Snowflake Task
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward

Activation Date	2025-Q1	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 7.2 — Tier 1 freshness SLA ≤ 1 hour | Section 10.3 — Snowflake Alert pattern	**Tags**	timeliness, freshness, sla, fact_order, tier-1, critical

Business Rule	fact_order must contain records ingested within the last 60 minutes. A lag exceeding this threshold means BI dashboards are showing stale order data, real-time operational decisions are based on outdated information, and the Tier 1 SLA contract with the Commerce Data Owner is in breach. This rule runs continuously, not just at pipeline time.
Pass Condition	**DATEDIFF('minute', MAX(audit_insertion_timestamp), CURRENT_TIMESTAMP()) ≤ 60**
Fail Action	**Immediately alert Data Steward via Slack #data-incidents. Page on-call engineer via PagerDuty. Halt downstream dashboard refresh. Initiate ingestion pipeline investigation. Acknowledge within 15 min, resolve within 4 hours per Tier 1 SLA.**
Notes	v1.2: Added SLA_WARNING band at 45 min (15 min before breach) for proactive alerting before the SLA is actually broken.

```
-- DQR-009: Freshness SLA check (Snowflake Task, every 10 min)
SELECT
    MAX(audit_insertion_timestamp)                          AS last_loaded_at,
    CURRENT_TIMESTAMP()                                     AS checked_at,
    DATEDIFF('minute',
        MAX(audit_insertion_timestamp),
        CURRENT_TIMESTAMP())                                AS lag_minutes,
    CASE
        WHEN DATEDIFF('minute', MAX(audit_insertion_timestamp),
             CURRENT_TIMESTAMP()) > 240  THEN 'CRITICAL_BREACH'
        WHEN DATEDIFF('minute', MAX(audit_insertion_timestamp),
             CURRENT_TIMESTAMP()) > 60   THEN 'SLA_BREACH'
        WHEN DATEDIFF('minute', MAX(audit_insertion_timestamp),
             CURRENT_TIMESTAMP()) > 45   THEN 'SLA_WARNING'
        ELSE 'WITHIN_SLA'
    END                                                     AS freshness_status
FROM commerce_prod.marts.fact_order;

-- Expected: freshness_status = 'WITHIN_SLA'
```

DQR-010 Country Code in Reference List

dim_customer | marts / Gold | Tier 1

MEDIUM
Status: Active
v1.0

DQ Dimension	Validity	**Test Case Ref**	TC-010
Primary Tool	dbt: relationships (to ref_country)	**Secondary Tool**	GE: expect_column_values_to_be_in_set
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q1	**Review Date**	2025-Q4 (Biannual)

Framework Refs	Section 4.4 — _code: VARCHAR, enum-like | Section 5 — accepted_values pattern	**Tags**	validity, dim_customer, country_code, reference-data, tier-1

Business Rule	country_code must be a valid ISO-3166-1 alpha-2 code present in the reference.ref_country lookup table. Invalid country codes break geographic segmentation, tax rule application, and regulatory reporting by jurisdiction.
Pass Condition	**LEFT JOIN ref_country WHERE ref.country_code IS NULL → returns 0 rows**
Fail Action	**Log warning. Flag invalid rows. Route to Data Steward review. Do not halt pipeline. Create Medium ticket. Update source system documentation if new valid code needed.**
Notes	If a new valid country code needs to be added, the process is: Steward approves → ref_country updated → rule automatically passes. Do not modify the rule to accommodate unvalidated codes.

```
-- DQR-010: Country code reference validation
SELECT  c.customer_id,
        c.country_code,
        c.audit_batch_id
FROM    commerce_prod.marts.dim_customer AS c
LEFT JOIN commerce_prod.reference.ref_country AS r
       ON c.country_code = r.country_code
WHERE   r.country_code IS NULL;

-- Expected: 0 rows
```

DQR-011 Zero-Amount Confirmed Order Detection

fact_order | marts / Gold | Tier 1

MEDIUM
Status: Active
v1.0

DQ Dimension	Validity	**Test Case Ref**	TC-011
Primary Tool	dbt: expression_is_true (order_amount > 0 for active statuses)	**Secondary Tool**	GE: custom conditional expectation
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q2	**Review Date**	2025-Q4 (Biannual)
Framework Refs	Section 5 — _amount CHECK >= 0 | Section 8 — Extended catalog	**Tags**	validity, fact_order, order_amount, zero, business-rule, tier-1

Business Rule	A confirmed, shipped, or delivered order with order_amount = 0.00 is a business anomaly — not a valid state. Pending orders may have zero amounts during creation. Active-status orders with zero revenue silently suppress revenue totals and distort average order value KPIs. This rule is a business logic extension of DQR-006 (which catches negatives).
Pass Condition	**COUNT(*) WHERE order_amount = 0 AND order_status_code IN ('confirmed','shipped','delivered') → returns 0**

Fail Action	**Log warning. Flag affected records. Alert Data Steward. Do not quarantine without Steward review — some zero-amount promotions may be valid business decisions requiring an exception.**
Notes	Business exceptions (e.g. 100% discount promotions) require a formal exception request per Section 3.4. The exception is logged in the exception registry, not in this rule.

```
-- DQR-011: Zero-amount active order detection
SELECT  order_id,
        order_amount,
        order_status_code,
        audit_insertion_timestamp
FROM    commerce_prod.marts.fact_order
WHERE   order_amount = 0.0000
  AND   order_status_code IN ('confirmed','shipped','delivered');

-- Expected: 0 rows
-- Note: 'pending' status with zero amount is permitted (pre-pricing orders)
```

DQR-012 Order Status Domain Validation

fact_order | marts / Gold | Tier 1

HIGH
Status: Active
v1.0

DQ Dimension	Validity	**Test Case Ref**	TC-012
Primary Tool	dbt: accepted_values(['pending','confirmed','shipped','delivered','cancelled'])	**Secondary Tool**	GE: expect_column_values_to_be_in_set
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q2	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 4.4 — _code: accepted_values via dbt | Section 5 — domain constraint	**Tags**	validity, fact_order, order_status_code, domain, tier-1

Business Rule	order_status_code must belong to the approved five-value domain. Values outside this set (e.g. 'PROCESSING', 'complete', 'NEW') are indicators of source system schema drift, a broken ETL transformation, or an unapproved status introduced without governance review. They break funnel analysis, churn calculation, and revenue-by-status reporting.
Pass Condition	**COUNT(*) WHERE order_status_code NOT IN approved set OR IS NULL → returns 0**
Fail Action	**Quarantine affected rows. Alert Steward. Block mart_sales_summary refresh. Open High ticket. If a new valid status is being introduced, it must follow the Change Management process (Section 3.3) before being added to the approved domain.**
Notes	Domain expansion requires: Steward approval + Owner sign-off + 14-day consumer notice + schema.yml update + GE suite update. Never add values informally.

```
-- DQR-012: Order status domain validation
SELECT  order_id,
        order_status_code,
        COUNT(*) AS occurrences
FROM    commerce_prod.marts.fact_order
WHERE   order_status_code NOT IN (
            'pending','confirmed','shipped','delivered','cancelled'
        )
   OR   order_status_code IS NULL
GROUP BY order_id, order_status_code
ORDER BY occurrences DESC;

-- Expected: 0 rows
-- Approved domain: pending | confirmed | shipped | delivered | cancelled
```

DQR-013 Mandatory Audit Column Completeness

All stg_, int_, dim_, fact_, mart_ tables | Silver / Gold | Tier 1+2

CRITICAL
Status: Active
v1.0

DQ Dimension	Completeness	**Test Case Ref**	TC-013
Primary Tool	dbt: not_null on all audit columns in every model	**Secondary Tool**	sqlfluff: column presence check at CI | Snowflake Task
Data Owner	Data Quality Lead	**Data Steward**	All Domain Stewards
Activation Date	2025-Q2	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 4.7 — Mandatory audit columns | Section 7.3 — 0% null tolerance on NOT NULL columns	**Tags**	completeness, audit-columns, lineage, all-tables, framework-compliance

Business Rule	All non-raw tables must contain audit_insertion_timestamp, audit_batch_id, source_system_id, and record_hash with zero null values. These columns are the backbone of lineage tracing, incident investigation, and pipeline idempotency. A null audit_batch_id makes it impossible to trace which pipeline run produced a given record.
Pass Condition	**All null counts for audit_insertion_timestamp, audit_batch_id, source_system_id, record_hash = 0**
Fail Action	**Halt pipeline (any table in Silver or Gold). This is a deployment blocker — the absence of mandatory audit columns makes the table non-compliant with the governance framework. Raise Critical ticket. Pipeline cannot resume until columns are populated.**
Notes	Applies universally. No exceptions are permitted for audit column completeness — these are the minimum viable governance columns for any managed data asset.

```
-- DQR-013: Audit column completeness (run per table, shown for fact_order)
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN audit_insertion_timestamp IS NULL THEN 1 ELSE 0 END)
                     AS null_insertion_ts,
    SUM(CASE WHEN audit_batch_id IS NULL            THEN 1 ELSE 0 END)
                     AS null_batch_id,
    SUM(CASE WHEN source_system_id IS NULL          THEN 1 ELSE 0 END)
                     AS null_source_id,
```

```
    SUM(CASE WHEN record_hash IS NULL               THEN 1 ELSE 0 END)
                      AS null_record_hash
FROM commerce_prod.marts.fact_order;

-- Expected: all null counts = 0
```

DQR-014 Record Hash Uniqueness

fact_order, dim_customer | Gold | Tier 1+2

HIGH
Status: Active
v1.0

DQ Dimension	Uniqueness	**Test Case Ref**	TC-014
Primary Tool	dbt: unique test on record_hash	**Secondary Tool**	Snowflake Task
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q2	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 4.7 — record_hash: SHA-256 of business keys for deduplication | Section 7.3	**Tags**	uniqueness, record_hash, deduplication, fact_order, dim_customer, tier-1

Business Rule	record_hash is the SHA-256 fingerprint of the business key columns, used as a second-layer uniqueness guarantee independent of the surrogate or natural PK. If two rows share an identical hash, they represent the same business event — the deduplication stage failed to eliminate a duplicate. This rule acts as the safety net below DQR-002.
Pass Condition	**GROUP BY record_hash HAVING COUNT(*) > 1 → returns 0 rows**
Fail Action	**Quarantine duplicate records. Open High incident. Trigger deduplication re-run. Investigate whether the source is emitting identical events or whether the hash computation is incorrectly excluding discriminating columns.**
Notes	Hash collision (two different records producing the same SHA-256) is theoretically possible but astronomically unlikely. Any apparent collision should first be investigated as a deduplication failure, not a hash collision.

```
-- DQR-014: Record hash uniqueness
SELECT  record_hash,
        COUNT(*) AS hash_count
FROM    commerce_prod.marts.fact_order
GROUP BY record_hash
HAVING  COUNT(*) > 1
ORDER BY hash_count DESC;

-- Expected: 0 rows
```

DQR-015 Daily Volume Anomaly Detection — fact_order

fact_order | marts / Gold | Tier 1

HIGH
Status: Active
v1.0

DQ Dimension	Timeliness + Completeness	**Test Case Ref**	TC-015
Primary Tool	Snowflake Task (daily 07:00 UTC)	**Secondary Tool**	GE: expect_table_row_count_to_be_between
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q2	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 7.2 — Tier 1: alert if ±20% of 7-day average | Section 9.1 severity model	**Tags**	timeliness, volume, anomaly, fact_order, tier-1, monitoring

Business Rule	The daily order count in fact_order must stay within ±20% of the 7-day rolling average. A sudden drop signals an ingestion failure or source system outage. A sudden spike signals a potential runaway job, data duplication, or load replay. Both are pipeline health signals requiring immediate investigation.
Pass Condition	**ABS(today_count - 7d_avg) / 7d_avg ≤ 0.20**
Fail Action	**Alert Data Steward via Slack #data-quality. Open High Jira ticket. Investigate pipeline logs and source system health. Do not halt pipeline — volume anomaly alone does not constitute a data correctness failure, but it mandates immediate investigation.**
Notes	The ±20% threshold is the Tier 1 default per Section 7.2. The Data Owner may request a tighter threshold (e.g. ±10%) for specific high-visibility datasets. Threshold changes require a rule version update.

```
-- DQR-015: 7-day rolling volume anomaly detection
WITH daily_counts AS (
    SELECT  order_date,
            COUNT(*) AS daily_count
    FROM    commerce_prod.marts.fact_order
    WHERE   order_date >= DATEADD('day', -8, CURRENT_DATE())
    GROUP BY order_date
),
baseline AS (
    SELECT AVG(daily_count) AS avg_7d
    FROM   daily_counts
    WHERE  order_date < CURRENT_DATE()
)
SELECT
    d.order_date,
    d.daily_count,
    ROUND(b.avg_7d, 0)                                  AS baseline_7d_avg,
    ROUND(ABS(d.daily_count - b.avg_7d)
          / NULLIF(b.avg_7d, 0) * 100, 2)              AS pct_deviation,
    CASE
        WHEN ABS(d.daily_count - b.avg_7d)
             / NULLIF(b.avg_7d, 0) > 0.20              THEN 'ANOMALY'
        ELSE 'WITHIN_BOUNDS'
    END                                                 AS volume_status
FROM daily_counts d CROSS JOIN baseline b
WHERE d.order_date = CURRENT_DATE();

-- Expected: volume_status = 'WITHIN_BOUNDS'
```

DQR-016 Surrogate Key Integrity — dim_customer

dim_customer | marts / Gold | Tier 1

HIGH
Status: Active
v1.0

DQ Dimension	Uniqueness	**Test Case Ref**	TC-016
Primary Tool	dbt: not_null + unique on customer_sk	**Secondary Tool**	GE: expect_column_values_to_not_be_null
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q2	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 4.4 — _sk: NUMBER IDENTITY, NOT NULL | Section 4.3 — dim_ requires _sk	**Tags**	uniqueness, surrogate-key, dim_customer, scd, tier-1

Business Rule	customer_sk is the IDENTITY-generated surrogate key used by all fact tables to join to dim_customer. A null or duplicate surrogate key breaks SCD Type 2 management and corrupts every fact-to-dimension join in the Gold layer. The surrogate key is platform-generated — a null indicates the IDENTITY column failed or was bypassed.
Pass Condition	**null_sk_count = 0 AND duplicate_sk_count = 0**
Fail Action	**Quarantine affected records. Open High incident. Investigate IDENTITY column configuration. Do not allow any fact table loading to proceed until SK integrity is confirmed — all fact joins will produce incorrect results.**
Notes	Do not confuse customer_sk (surrogate, system-generated) with customer_id (natural key, source-system). Both must be unique, but they serve different purposes. DQR-001/002 cover the natural key.

```
-- DQR-016: Surrogate key null and duplicate check
SELECT
    SUM(CASE WHEN customer_sk IS NULL THEN 1 ELSE 0 END)
                AS null_sk_count,
    COUNT(*) - COUNT(DISTINCT customer_sk)
                AS duplicate_sk_count
FROM commerce_prod.marts.dim_customer;

-- Expected: null_sk_count = 0  AND  duplicate_sk_count = 0
```

DQR-017 Empty String and Whitespace Detection

dim_customer, fact_order (string columns) | Gold | Tier 1+2

MEDIUM
Status: Active
v1.0

DQ Dimension	Completeness + Validity	**Test Case Ref**	TC-017
Primary Tool	dbt: custom SQL test (TRIM check)	**Secondary Tool**	GE: expect_column_values_to_match_regex (^\S.*\S$)
Data Owner	Data Quality Lead	**Data Steward**	All Domain Stewards

Activation Date	2025-Q2	**Review Date**	2025-Q4 (Biannual)
Framework Refs	Section 7.1 — Completeness not limited to IS NULL | Section 6.1 — data type correctness	**Tags**	completeness, validity, empty-string, whitespace, dim_customer, tier-1

Business Rule	Empty strings ('') and whitespace-only values (' ') pass IS NOT NULL checks but are semantically nulls. They corrupt lookup joins (matching ' ' to nothing), inflating miss-rates, and break string operations that assume meaningful content. This rule is the second half of completeness coverage, paired with all not_null rules.
Pass Condition	**COUNT(*) WHERE TRIM(customer_name) = '' → returns 0**
Fail Action	**Log warning. Flag records. Route to Data Steward review. Do not auto-correct — an empty string may need to be treated as NULL (and trigger upstream data re-pull) rather than replaced with a default value.**
Notes	Apply this rule to all VARCHAR columns that have a corresponding not_null rule. Priority columns: customer_name, order_status_code, source_system_id, country_code.

```
-- DQR-017: Empty and whitespace-only string detection (dim_customer example)
SELECT  customer_id,
        customer_name,
        LENGTH(customer_name)       AS raw_length,
        LENGTH(TRIM(customer_name)) AS trimmed_length
FROM    commerce_prod.marts.dim_customer
WHERE   TRIM(customer_name) = ''
   OR   customer_name = '';

-- Expected: 0 rows
-- Apply same pattern to: source_system_id, country_code, customer_segment_code
```

DQR-018 SCD Type 2 Currency Flag Integrity

dim_customer | marts / Gold | Tier 1

HIGH
Status: Active
v1.0

DQ Dimension	Validity + Consistency	**Test Case Ref**	TC-018
Primary Tool	dbt: custom SQL test (group by customer_id)	**Secondary Tool**	Snowflake Task
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q2	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 4.3 — dim_: is_current_flag required | Section 4.4 — _flag: BOOLEAN, positive logic	**Tags**	validity, consistency, scd, is_current_flag, dim_customer, tier-1

Business Rule	Per SCD Type 2 semantics, exactly one record per customer natural key must have is_current_flag = TRUE at any point in time. Multiple current records means the SCD process produced a split-brain state. Zero current records means the customer is orphaned from all

	current-period joins. Both conditions corrupt all BI queries that filter WHERE is_current_flag = TRUE.
Pass Condition	**Multiple current records per customer_id = 0. Customers with no current record = 0.**
Fail Action	**Quarantine all affected customer_ids. Open High incident. Trigger SCD repair job. Block dim_customer downstream consumption until repaired. Alert Data Owner if > 5% of customers are affected.**
Notes	Two separate SQL checks required (see validation SQL). Both must pass independently.

```
-- DQR-018a: Multiple current records per customer (split-brain)
SELECT  customer_id,
        COUNT(*) AS current_record_count
FROM    commerce_prod.marts.dim_customer
WHERE   is_current_flag = TRUE
GROUP BY customer_id
HAVING  COUNT(*) > 1;

-- Expected: 0 rows

-- DQR-018b: Customers with NO current record (orphaned)
SELECT  customer_id
FROM    commerce_prod.marts.dim_customer
GROUP BY customer_id
HAVING  SUM(CASE WHEN is_current_flag = TRUE THEN 1 ELSE 0 END) = 0;

-- Expected: 0 rows
-- Both 018a and 018b must return 0 rows for this rule to PASS
```

DQR-019 Staging-to-Curated Row Count Reconciliation

stg_order → fact_order | Silver → Gold | Tier 2

HIGH
Status: Active
v1.0

DQ Dimension	Consistency + Completeness	**Test Case Ref**	TC-019
Primary Tool	dbt: custom post-hook test	**Secondary Tool**	Snowflake Task (post-pipeline run)
Data Owner	Commerce Domain VP	**Data Steward**	Commerce Data Steward
Activation Date	2025-Q2	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 4.3 — layered architecture integrity | Section 13.2 — pipeline audit trail	**Tags**	consistency, completeness, reconciliation, stg_order, fact_order, pipeline, tier-2

Business Rule	After each pipeline run, the row count promoted to fact_order must be ≤ the count in stg_order for the same audit_batch_id (deduplication reduces, never inflates). If fact_order contains MORE rows than stg_order for a given batch, records were inserted outside the controlled pipeline — a governance and integrity violation. An empty load (zero records promoted) is also a failure signal.
Pass Condition	**fact_count ≤ stg_count AND fact_count > 0**

Fail Action	**INTEGRITY_BREACH: halt pipeline, Critical alert — records inserted outside controlled pipeline. EMPTY_LOAD: High alert — investigate ingestion and source system for batch delivery failure.**
Notes	This rule runs per-batch using audit_batch_id as the correlation key. It requires that audit_batch_id is populated in both source and target tables — which DQR-013 guarantees.

```
-- DQR-019: Staging-to-curated reconciliation (per batch_id)
WITH counts AS (
    SELECT
        (SELECT COUNT(*)
         FROM   commerce_prod.staging.stg_order
         WHERE  audit_batch_id = :current_batch_id)  AS stg_count,
        (SELECT COUNT(*)
         FROM   commerce_prod.marts.fact_order
         WHERE  audit_batch_id = :current_batch_id)  AS fact_count
)
SELECT
    stg_count,
    fact_count,
    stg_count - fact_count                           AS dedup_reduction,
    CASE
        WHEN fact_count > stg_count  THEN 'INTEGRITY_BREACH'
        WHEN fact_count = 0          THEN 'EMPTY_LOAD'
        ELSE 'RECONCILED'
    END                                              AS reconciliation_status
FROM counts;

-- Expected: reconciliation_status = 'RECONCILED'
```

DQR-020 Schema Drift Detection

All Tier 1 & 2 tables | Gold + Silver | Tier 1+2

CRITICAL
Status: Active
v1.0

DQ Dimension	Schema Compliance	**Test Case Ref**	TC-020
Primary Tool	dbt compile (detects undeclared columns) | Snowflake INFORMATION_SCHEMA diff	**Secondary Tool**	CI/CD PR lineage check
Data Owner	Data Quality Lead	**Data Steward**	All Domain Stewards
Activation Date	2025-Q2	**Review Date**	2025-Q3 (Quarterly)
Framework Refs	Section 7.3 — schema drift: Critical alert + pipeline pause regardless of tier | Section 3.3 — Change Management	**Tags**	schema, drift, compliance, all-tables, framework-compliance, breaking-change

Business Rule	Any unplanned deviation between the live table schema and the schema declared in dbt schema.yml constitutes drift. This includes added columns (even nullable ones break GE expectations enumerating columns), missing columns, and data type changes. Schema drift is the most common source of silent downstream breakage — consumers may continue to run but return incorrect results.

Pass Condition	**FULL OUTER JOIN declared vs. live schema → 0 rows with drift_status ≠ 'OK'**
Fail Action	**Critical alert immediately. Halt pipeline. Page on-call Custodian. Determine whether change was intentional (missed Change Management process) or accidental (deployment error). If intentional: open retrospective Change Request ticket retroactively. If accidental: rollback and investigate.**
Notes	This is the only rule that triggers a Critical halt for Medium-severity schema additions (new nullable columns). Even non-breaking changes bypass the Change Management process when they are unplanned.

```
-- DQR-020: Schema drift detection (fact_order example)
WITH declared AS (
    -- Sourced from dbt manifest.json column inventory
    SELECT column_name, data_type
    FROM   governance.schema_baseline
    WHERE  table_name = 'fact_order'
),
live_schema AS (
    SELECT LOWER(column_name) AS column_name,
           LOWER(data_type)   AS data_type
    FROM   information_schema.columns
    WHERE  table_schema = 'MARTS'
      AND  table_name   = 'FACT_ORDER'
)
SELECT
    COALESCE(d.column_name, l.column_name)  AS column_name,
    d.data_type                             AS declared_type,
    l.data_type                             AS live_type,
    CASE
        WHEN d.column_name IS NULL THEN 'COLUMN_ADDED_UNDECLARED'
        WHEN l.column_name IS NULL THEN 'COLUMN_MISSING_FROM_LIVE'
        WHEN d.data_type <> l.data_type THEN 'DATA_TYPE_MISMATCH'
        ELSE 'OK'
    END                                     AS drift_status
FROM declared d
FULL OUTER JOIN live_schema l ON d.column_name = l.column_name
WHERE COALESCE(
    CASE WHEN d.column_name IS NULL THEN 1 END,
    CASE WHEN l.column_name IS NULL THEN 1 END,
    CASE WHEN d.data_type <> l.data_type THEN 1 END
) = 1;

-- Expected: 0 rows
```

5. New Rule Intake Template

Any engineer or Data Steward may propose a new DQ rule using this template. Completed templates are submitted via Jira (project: DATA-GOVERNANCE, issue type: DQ Rule Proposal). The Data Steward assigns the next sequential DQR-XXX ID upon acceptance of the proposal.

Field	Instructions & Requirements
Rule ID	Leave blank — assigned sequentially by Data Steward upon acceptance
Rule Name	Short descriptive title, maximum 60 characters. Start with the subject (column or entity), then the condition. Example: 'Customer Email Format Validation'.
Target Table	Fully qualified: schema.table_name (e.g. commerce_prod.marts.fact_order)
DQ Dimension	One of: Completeness | Uniqueness | Validity | Consistency | Timeliness | Referential Integrity | Schema Compliance
Severity	Critical — pipeline halt | High — quarantine + alert | Medium — log + flag | Low — log only. Justify your choice.
Business Rule	Plain language description of what constitutes a violation and why it matters to the business. Minimum 2 sentences.
Validation SQL	Complete, executable SQL returning 0 rows on PASS. Include both the validation query and the expected result comment.
Pass Condition	One sentence: exactly what the query must return to pass. Example: 'COUNT(*) = 0 when grouped by order_id HAVING COUNT(*) > 1'.
Fail Action	Exact action: Halt pipeline | Alert role | Quarantine rows | Log only. Specify which role is alerted and via which channel.
Primary Tool	dbt (test type) | Great Expectations (expectation name) | Snowflake Task | Databricks DLT | CI/CD
Secondary Tool	Optional second enforcement layer
Data Owner	Named individual with business accountability for this rule
Data Steward	Named individual responsible for rule maintenance and review
Proposed Activation	Target date or sprint for activation
Review Frequency	Quarterly (Critical/High) | Biannual (Medium) | Annual (Low)
Related Rules	Any existing rules this rule complements, extends, or supersedes (DQR-XXX)
Tags	Comma-separated: table, dimension type, feature area, tier

5.1 Rule Naming Convention

Rule names follow a consistent pattern to make the catalog scannable and the nature of each rule immediately clear without reading the description.

Pattern	Example	Anti-Pattern (Prohibited)
<Subject> <Condition>	Order Amount Non-Negative	'Amount check' — too vague
<Subject> <Type> Completeness	Customer Name Completeness	'Customer data quality' — not specific
<FK Relationship> Integrity	FK Integrity — fact_order → dim_customer	'Check FK' — not descriptive
<Column> Domain Validation	Order Status Domain Validation	'Status values' — no structural signal
<Entity> Freshness SLA — <period>	Freshness SLA — fact_order (1-hour)	'Freshness check' — missing SLA value

6. Exception Registry

All active rule exceptions are logged here. An empty registry means all rules are being met without exception. This section is reviewed monthly by the Data Steward per the governance cadence (framework Section 13.4).

Exception ID	Rule	Affected Dataset	Justification	Compensating Control	Approved By	Expiry Date	Status
EXC-001	—	—	No active exceptions at catalog publication date	—	—	—	**None**

New exceptions are added to this table upon Steward/Owner approval. Expired exceptions are moved to the Deprecated Exceptions archive (Section 7) and replaced with a remediation ticket reference.

7. Deprecated Rules Archive

Rules retired from active enforcement are preserved here. The SQL, rationale, and retirement context are maintained permanently as a governance audit trail.

Rule ID	Rule Name	Retired Date	Reason for Deprecation	Superseded By
—	No deprecated rules at catalog publication date	—	—	—

8. Programme Roadmap — Document 4 in Context

The Rules Catalog is the operational bridge between the Test Plan (which proves rules work) and the Monitoring document (which runs them continuously). Its authority derives from all three upstream documents being complete and signed off.

#	Deliverable	Status	Relationship to This Catalog
1	Database Coding & Naming Standards v2.0	**Complete**	Source of naming rules, data type constraints, and audit column requirements that all DQ rules enforce
2	Case Study — DG & DQ Transformation	**Complete**	Provides the real-world context, baseline data, and business justification behind each rule's existence
3	Data Quality Test Plan v1	**Complete**	Proves each rule is correctly implemented and detectable — every active rule in this catalog has a passing test case
4	Data Quality Rules Catalog ◀ this document	**Active**	Authoritative registry — what rules exist, who owns them, what they do, and whether they have exceptions
5	DQ Monitoring & DQS Dashboard	**Next**	Operationalises this catalog into a live dashboard: DQS per dataset, rule failure heatmap, SLA trend, alerting runbooks

8.1 Document 5 Preview — DQ Monitoring & DQS Dashboard

The final document in the programme will translate this catalog into running infrastructure. It will specify the complete Snowflake Task tree, the DQS calculation pipeline architecture, the Slack and PagerDuty integration design, the BI dashboard layout (DQS trends, rule failure heatmaps, volume anomaly charts), and the full incident response runbook per severity level.

Monitoring & DQS Dashboard — Scope Preview (Doc 5)
▸ Snowflake Task tree: full task graph with warehouse assignments, cron schedule, and dependency chain
▸ DQS pipeline architecture: nightly calculation → dq_results table → dashboard feed → historical trend
▸ Databricks DLT expectation summary: pass/drop/fail rates per pipeline run
▸ Slack alert routing: which rule failures go to which channel at which severity
▸ PagerDuty integration: Critical rules → on-call rotation → 15-min ack → 4-hr resolution ladder
▸ BI dashboard specification: DQS gauge, dimension breakdown, rule failure heatmap, 30-day trend ▸ Incident response runbook: per-severity investigation steps, query templates, escalation contacts ▸ Quarterly review checklist: catalog audit procedure, tier re-assessment, DQS trend analysis template